

constant feedback and adaptation to changes in requirements.

DevOps: This practice combines development and operations to ensure streamlined processes for software development, testing, and deployment.

Test-driven development: This involves writing automated tests before writing code, ensuring that code is thoroughly tested and meets all requirements.

Continuous integration and delivery: This practice involves regularly integrating code into a shared repository and deploying the latest version to production.

Microservices architecture: This involves breaking down an application into smaller, independent services, making it easier to manage and scale.

Containerisation: This practice involves packaging an application and its dependencies into a container, making it easy to deploy and run on any platform.

Cloud computing: This involves using cloud platforms like AWS or Azure for hosting and scaling applications, leveraging the power of the cloud for optimal performance and cost-effectiveness.

Code review: This involves having multiple developers review each other's code, ensuring that code is well-written, efficient, and adheres to coding standards.

By implementing these best and modern practices, developers can create high-quality software that meets the needs of their users and keeps pace with the rapidly changing technological landscape.

Best practices for developers

Use of agile methodologies

Agile methodologies incorporate a set of practices and values that prioritise flexibility, collaboration, and iterative development to quickly respond to changing requirements and

feedback. In software development, agile methodologies have become increasingly popular to improve team efficiency, reduce risk, and deliver high-quality products. Here are some of the best practices for utilising agile methodologies in software development.

Team collaboration: Agile methodologies emphasise teamwork, open communication, and collaboration. Team members from different disciplines work together to ensure that the requirements are appropriately scoped, development tasks are prioritised, and testing and deployment are coordinated.

Scrum: Scrum is an agile framework that helps with implementing these best practices. It consists of a product owner, scrum master, and development team that works together within a fixed timeframe (sprint) to deliver the product. The scrum team holds regular meetings (daily stand-ups, sprint planning, sprint reviews) to manage workflows, assess progress, and identify ways to improve.

User feedback: Agile methodologies prioritise user feedback and testing from the project's inception. This feedback is used to inform product development teams, ensure user satisfaction, and reduce waste when addressing defects.

Flexible planning: Agile planning is adaptive and iterative. Workflows, tasks, and features are continually refined and optimised based on user feedback and changing requirements. The agile process enables a team to respond effectively to shifting business needs without causing any significant delay in development.

Continuous integration and delivery: Agile methodologies emphasise continuous integration and delivery. This practice involves frequently testing and integrating new code into the main codebase and deploying changes to the production environment. This process helps reduce risk and mitigate potential production issues.

Data-driven decision making: Agile teams rely on data to make informed

decisions about the development process. The use of data encompasses everything from user feedback collected through A-B testing, metrics on feature usage, error rates, and even process improvement measures to ensure that the team is delivering constant value.

Overall, agile methodologies are an excellent way for developers to improve their best practices. They help the team collaborate, organise and deliver value with speed and agility while mitigating risk.

Collaboration with non-technical stakeholders

Collaboration with non-technical stakeholders refers to the process of working together with individuals who are not IT professionals or experts in the technical aspect of a project. These stakeholders are typically involved in decision-making, planning, and implementation of a project. They may include business owners, clients, end users, or any other stakeholder who is not technical but has a significant impact on the project.

Effective collaboration with non-technical stakeholders requires clear communication, active listening, and a thorough understanding of their vision, goals, and requirements.

One of the key benefits of collaborating with non-technical stakeholders is that it provides a diverse perspective and can help identify blind spots in the technical team.

To collaborate effectively with non-technical stakeholders, technology teams should:

- Establish clear communication channels and protocols
- Identify the stakeholders' needs, goals, and priorities
- Develop a shared understanding of technical terminology and concepts
- Provide regular updates on project progress and milestones
- Create opportunities for stakeholder feedback and input

By working with non-technical